Das O-Kalkül (Landau-Symbole)

- Laufzeitfunktionen können sehr kompliziert sein.
- Zur Bestimmung der Komplexität eines Problems müssen Laufzeitfunktionen bewertet und verglichen werden.
- Entscheidend ist das asymptotischen Wachstums einer Funktion.
- **Idee:** Vergleich des Wachstums zweier Funktionen

  $g(n)$ und $f(n)$ für große n.

  - „Welche Funktion gewinnt das Rennen?“
- Vorteile:
  - Konstante Faktoren sind irrelevant.
  - Bei Polynomen und Potenzen zählt nur der größte Exponent.



Quelle: Wikipedia

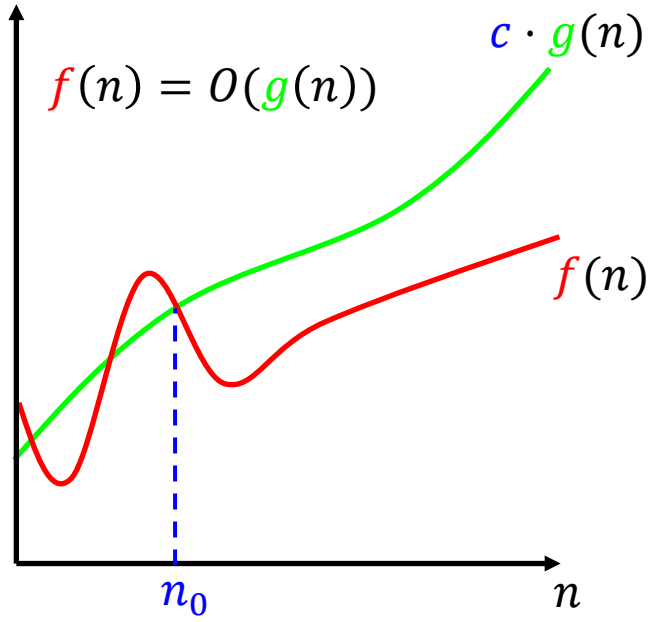
Edmund Landau, 1877-1938.

Definition: (sprich: „Groß-Oh“)

$O(g(n)) = \{f(n)$: Es gibt positive Konstanten c und n_0, so dass

$0 \leq |f(n)| \leq c \cdot |g(n)|$ für alle $n \geq n_0\}$.

- Schreibweise: $f(n) = O(g(n))$ anstatt $f(n) \in O(g(n))$.
- Beispiele:
  - $3n \quad = O(n^2)$
  - $\frac{n^2}{2} - 3n = O(n^2)$
  - $6n^3 \quad \neq O(n^2)$
- $f(n) = O(g(n))$ entspricht „$a \leq b$“.

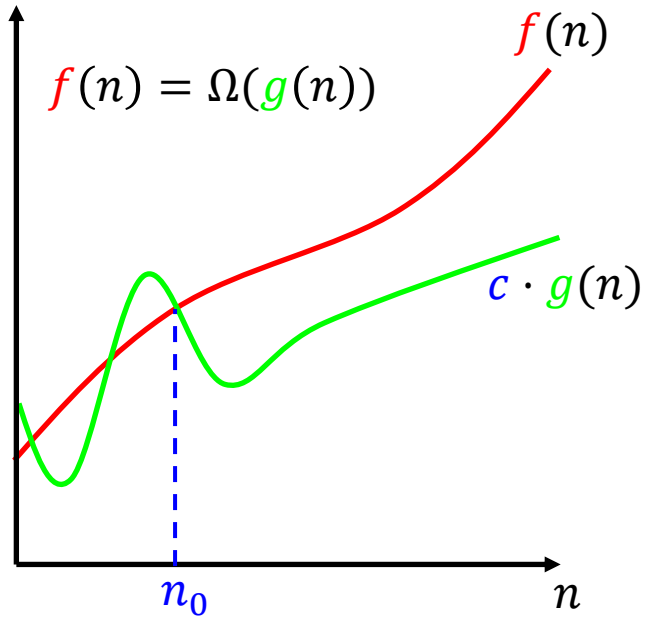


Definition: (sprich: „Groß-Omega“)

$\Omega(g(n)) = \{f(n)$: Es gibt positive Konstanten c und n_0, so dass

$0 \leq c \cdot |g(n)| \leq |f(n)|$ für alle $n \geq n_0\}$.

- Beispiele:
  - $3n \quad \neq \Omega(n^2)$
  - $\frac{n^2}{2} - 3n = \Omega(n^2)$
  - $6n^3 \quad = \Omega(n^2)$
- $f(n) = \Omega(g(n))$ entspricht „$a \geq b$“.

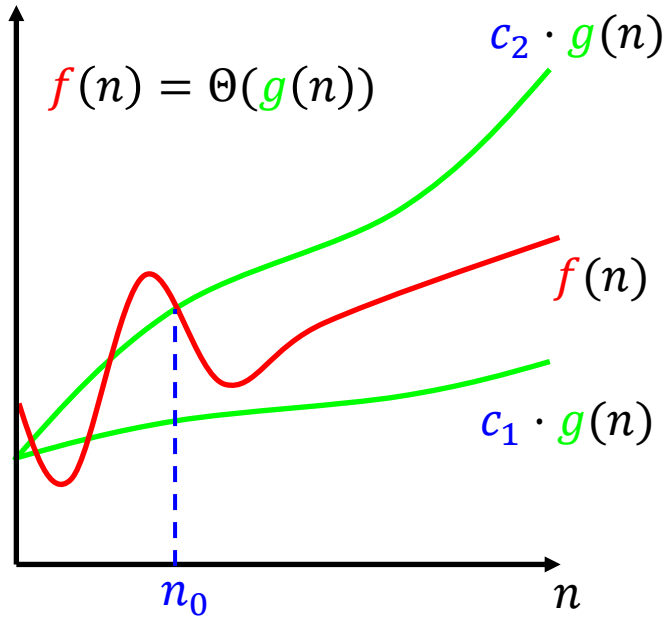


Definition: (sprich: „Groß-Theta“)

$\Theta(g(n)) = \{f(n)$: Es gibt positive Konstanten c_1, c_2 und n_0, so dass

$0 \leq c_1 \cdot |g(n)| \leq |f(n)| \leq c_2 \cdot |g(n)|$ für alle $n \geq n_0\}$.

- Beispiele:
  - $3n \quad \neq \Theta(n^2)$
  - $\frac{n^2}{2} - 3n = \Theta(n^2)$
  - $6n^3 \quad \neq \Theta(n^2)$
- $f(n) = \Theta(g(n))$ entspricht „$a = b$“.



§1.2 Komplexitätstheorie

Definition: (sprich: „Klein-Oh“)

$o(g(n)) = \{f(n)$: Für alle Konstanten $c > 0$ gibt es ein $n_0 > 0$, so dass $0 \leq |f(n)| < c\,|g(n)|$ für alle $n \geq n_0\}$.

- Beispiele:
  - $3n \quad = o(n^2)$
  - $\frac{n^2}{2} - 3n \neq o(n^2)$
  - $6n^3 \quad \neq o(n^2)$
- $f(n) = o(g(n))$ entspricht „$a < b$“.
- Äquivalente Bedingung für $f(n) = o(g(n))$:

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0.$$

§1.2 Komplexitätstheorie

Definition: (sprich: „Klein-Omega“)

$\omega(g(n)) = \{f(n)$: Für alle Konstanten $c > 0$ gibt es ein $n_0 > 0$, so dass $0 \leq c\,|g(n)| < |f(n)|$ für alle $n \geq n_0\}$.

- Beispiele:
  - $3n \neq \omega(n^2)$
  - $\frac{n^2}{2} - 3n \neq \omega(n^2)$
  - $6n^3 = \omega(n^2)$
- $f(n) = \omega(g(n))$ entspricht „$a > b$“.
- Äquivalente Bedingung für $f(n) = \omega(g(n))$:

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty.$$

Zusammenfassung:

- $O(g) \cap \Omega(g) = \Theta(g)$
- $O(g) \setminus \Theta(g) = o(g)$
- $\Omega(g) \setminus \Theta(g) = \omega(g)$
- $o(g) \subset O(g), \Theta(g) \subset O(g)$
- $\omega(g) \subset \Omega(g), \Theta(g) \subset \Omega(g)$

- $$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \begin{cases} 0 & \Leftrightarrow \quad f = o(g) \quad \Leftrightarrow \quad g = \omega(f) \\ \infty & \Leftrightarrow \quad f = \omega(g) \quad \Leftrightarrow \quad g = o\,(f) \\ \text{const} \neq 0, \infty & \Leftrightarrow \quad f = \Theta(g) \quad \Leftrightarrow \quad g = \Theta(f) \end{cases}$$

§1.3 Abschätzung der Laufzeit von Algorithmen

Beispiel: Binäre Suche

```
int search(int[] arr, int key)
{  // Voraussetzung: a ist aufsteigend sortiert
   int li = 0;
   int re = arr.length-1;
   while (re >= li) {
      int m = (li+re)/2;
      if (key == arr[m]) return m;
      if (key <  arr[m]) re = m-1;
      else               li = m+1;
   }
   return -1;
}
```

- Laufzeit der binären Suche
  - Best case: $O(1)$
    - Der Schlüssel wird bei der ersten `if`-Anfrage gefunden.

§1.4 NP-Probleme

- **Laufzeit**
  - Bei n verschiedenen Aussagevariablen gibt es 2^n verschiedene Wahrheitswertebelegungen.
  - Für eine gegebene Wahrheitswertbelegung kann in polynomialer Laufzeit getestet werden, ob er den Ausdruck erfüllt oder nicht.
- Laufzeit zur Berechnung der Entscheidung, ob SAT lösbar: $O(2^n)$
- Es ist unbekannt, ob es einen effizienteren Algorithmus gibt.
- ▶ Die Komplexität ist unbekannt.

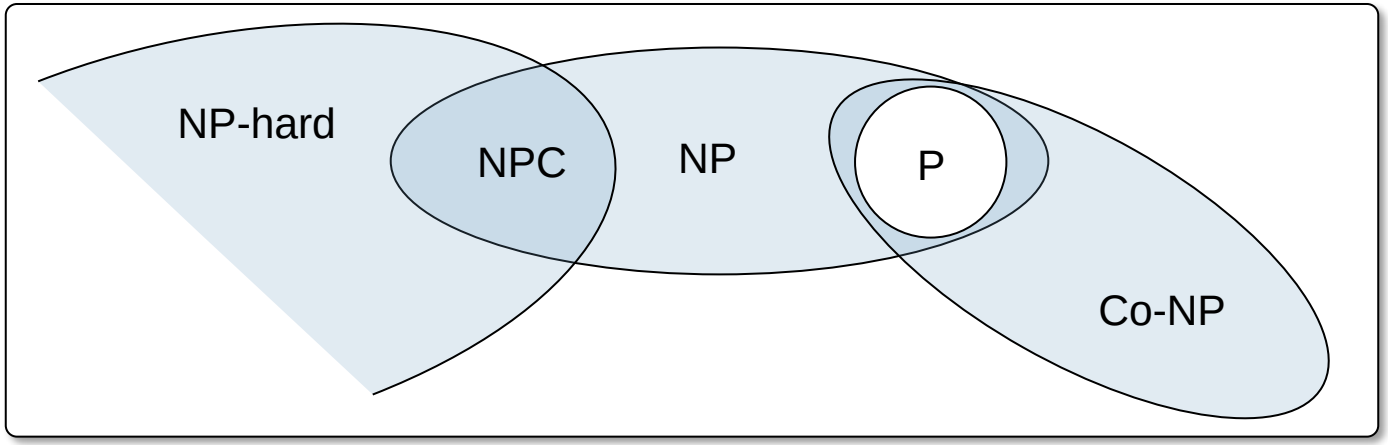
- **Aber:** Eine Lösung von SAT kann leicht verifiziert werden:
  - Ist eine Wertebelegung eine Lösung des aussagenlogischen Ausdrucks?
- Laufzeitenvergleich:
  - **Berechnung** einer Lösung von SAT: $O(2^n)$
  - **Verifikation** einer Lösung von SAT: $O(n^k) \subset o(2^n)$, für ein $k \in \mathbb{N}$.

§1.4 NP-Probleme

- Alle NP-vollständigen Probleme sind gleich schwierig:
- Ein polynomialer Algorithmus für ein einziges NPC-Problem würde P = NP beweisen!

Gibt es für ein NP-vollständiges Problem einen polynomialen Algorithmus, gibt es für alle NP-vollständigen Probleme einen polynomialen Algorithmus.

- Für alle NPC Probleme sind aber nur exponentielle Algorithmen bekannt.
  - Es ist nicht bekannt, ob es für NPC Probleme polynomiale Algorithmen gibt.



§1.4 NP-Probleme

- Lösungsansätze für schwierige Probleme
  - **Randomisierte Algorithmen**: Der Ablauf wird zufällig gesteuert.
    - **Las-Vegas-Algorithmen**:
      - Liefern immer das richtige Ergebnis.
      - Der Erwartungswert der Laufzeit ist polynomial, der worst case aber exponentiell.
    - **Monte-Carlo-Algorithmen**:
      - Liefern nur zu einer vorgegebenen Wahrscheinlichkeit eine korrekte Lösung.
      - Die Laufzeit ist im worst case (i.d.R.) polynomial.
  - **Approximative Algorithmen**:
    - Die Lösung ist nicht optimal, aber innerhalb einer vorgegebenen Toleranz.
    - Die Laufzeit ist im worst case polynomial.

Mathematisches Glossar

- Die **Summe** der Folgenglieder einer Folge $a_1, a_2, a_3, \ldots$ von a_m bis a_n, $m \leq n$, wird abgekürzt als

$$\sum_{i=m}^{n} a_i = a_m + a_{m+1} + \cdots + a_n.$$

- Spezielle Summen:

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2},$$

$$\sum_{i=1}^{n} i^2 = 1 + 4 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6},$$

$$\sum_{i=0}^{n} q^i = \frac{1 - q^{n+1}}{1 - q}.$$

Mathematisches Glossar

- Die n-te **Potenz** von a: $a^n = \underbrace{a \cdot a \cdot \ldots \cdot a}_{n}$
  - $a \in \mathbb{R} \setminus \{0\}$ heißt Basis
  - $n \in \mathbb{N}$ heißt Exponent
- Spezielle Exponenten: $a^0 = 1$ $(0^0 := 1)$

  $a^1 = a$
- Negative Exponenten: $a^{-n} = \frac{1}{a^n}$
- Rationale Exponenten $n, m \in \mathbb{N}$: $a^{\frac{m}{n}} = \sqrt[n]{a^m} = \left(\sqrt[n]{a}\right)^m$
- Produkt von Potenzen $x, y \in \mathbb{R}$: $a^x \cdot a^y = a^{x+y}$
- Potenz von Potenzen $x, y \in \mathbb{R}$: $(a^x)^y = a^{x \cdot y}$

Mathematisches Glossar

- **Logarithmus** von $a > 0$ zur Basis $b > 0$: $\log_b a$
  - Derjenige Exponent, mit dem man b potenzieren muss, um a zu erhalten, d.h. $b^x = a \Leftrightarrow x = \log_b a$
- Spezielle Werte: $\log_b 1 = 0,\ \log_b b = 1$
- Spezielle Logarithmen:
  - Dekadischer Logarithmus (Basis 10): $\lg a = \log_{10} a$
  - Natürlicher Logarithmus (Basis e): $\ln a = \log_e a$
  - Dualer Logarithmus (Basis 2): $\mathrm{ld}\, a = \mathrm{lb}\, a = \log_2 a$
- Umrechnung: $\log_b a = \frac{\ln a}{\ln b}$

Mathematisches Glossar

- **Ableitungen spezieller Funktionen**
  - Potenzfunktion ($a \neq 1$):
    $$\frac{d}{dx}(x^a) = a \cdot x^{a-1}$$
  - Logarithmusfunktion:
    $$\frac{d}{dx}\ln x = \frac{1}{x}$$
    $$\frac{d}{dx}\log_a x = \frac{1}{x \cdot \ln a}$$
  - Exponentialfunktion:
    $$\frac{d}{dx}e^x = e^x$$
    $$\frac{d}{dx}a^x = a^x \cdot \ln a$$

Mathematisches Glossar

- **Allgemeine Ableitungsregeln** für differenzierbare Funktionen f und g
  - Summenregel: $(f+g)' = f' + g'$
  - Produktregel: $(f \cdot g)' = f' \cdot g + f \cdot g'$
  - Quotientenregel ($g \neq 0$): $\left(\frac{f}{g}\right)' = \frac{f' \cdot g - f \cdot g'}{g^2}$
  - Kettenregel: $\big(f(g)\big)' = f'(g) \cdot g'$
  - Regel von de l'Hospital ($g' \neq 0$)

$$\lim_{x \to x_0} \frac{f'}{g'} = c \quad \Rightarrow \quad \lim_{x \to x_0} \frac{f}{g} = c$$

eine Konstante c und f, g konvergieren gegen 0 bzw. divergieren.

Bemerkung: Dabei ist $x_0 = \pm\infty$ und $c = \pm\infty$ erlaubt.

Mathematisches Glossar

- Bestimmung des **Erwartungswerts** E einer Zufallsvariable X:
  - **Werte:** Die Zufallsvariable X kann die Werte

    $$X_1, \dots, X_n$$

    annehmen.
  - **Wahrscheinlichkeitsverteilung:** Die Werte $X_1, \dots, X_n$ werden mit den Wahrscheinlichkeiten

    $$p_1, \dots, p_n$$

    angenommen.

    Dabei muss gelten: $\sum_{i=1}^{n} p_i = p_1 + \cdots + p_n = 1$.
  - Der Erwartungswert $E(X)$ berechnet sich als

    $$E(X) = \sum_{i=1}^{n} p_i \cdot X_i \,.$$